Betriebssysteme
4 – ÜB – Dateiverwaltung

Aufgabe 1: Dateien und Verzeichnisse

a) Welche der vier großen Probleme der Speicherverwaltung treten auch bei der Datei-Verwaltung auf? Begründen Sie Ihre Antwort!

b) Geben Sie ein Beispiel dafür an, dass relative Dateinamen nicht eindeutig sind.

c) Die Dateiendung ist nur eine Konvention. Wie hängt diese angebliche Konvention damit zusammen, dass das OS trotzdem richtig reagiert, wenn man auf eine Datei doppelklickt? Was passiert, wenn man sich nicht an die Konvention hält?

d) Beantworten Sie die folgenden Kurzfragen zu Dateiendungen und Dateiarten:

- Ist ein Microsoft-Word-Dokument (mit der Dateiendung `.docx`) eine Text- oder eine Binär-Datei? Begründen Sie Ihre Antwort!
- Wo ist gespeichert, ob eine Datei eine Text- oder eine Binärdatei ist?
- Ist jede Textdatei auch eine Binärdatei?
- Ist jede Binärdatei auch eine Textdatei?

e) Recherchieren Sie ein Beispiel aus einem echten OS für:

- eine zeichenorientierte Spezialdatei,
- eine blockorientierte Spezialdatei und
- eine Pseudo-Gerätedatei

und erklären Sie jeweils, wie diese Datei funktioniert.

f) Angenommen, Sie haben eine Datei und eine Verknüpfung zur Datei, ferner kennen Sie den Datei-Inhalt. Wie können Sie feststellen, welche Art von Verknüpfung vorliegt (Hard Link, slow oder fast Sym Link), wenn Sie dazu nur die Systemaufrufe `delete()` und `read()` zur Verfügung haben?

g) Schreiben Sie einen Algorithmus (in einer nicht-obskuren Programmiersprache Ihrer Wahl oder in Pseudo-Code) zur Lösung folgende Problemstellung:

Gegeben: Ein relativer Pfad zu einer existierenden Datei als Zeichenfolge,
z. B.: `..\..\test\hallo\..\ordner\.\hier.txt`

Gesucht: Der zugehörige, absolute, kanonische Pfad, also der eindeutige Pfad zur Datei, unter Berücksichtigung des aktuellen Arbeitsverzeichnisses und ohne die `..` und `.` für relative Pfade.

Hinweis: Der Algorithmus ist im Kern selbst zu schreiben. Einfach "nur" eine fertige Funktion der gewählten Sprache (z. B. aus einer Bibliothek) aufzurufen, genügt der Aufgabenstellung nicht.

Aufgabe 2: Dateien und Verzeichnisse

a) Was unterscheidet die Dateistrukturen (VL-Folie 12) von den Allokationsverfahren (VL-Folie 28 ff.)?

b) Warum lassen sich die Allokationsverfahren für Dateien nicht auch für den Arbeits-Speicher verwenden?

Hinweise:

- Wie lange dauert es, einen Zeiger in einer Tabelle zu verfolgen?
- Wie lange dauert eine einzelne Rechnung in einem Register?
- Wie lange dauert es, bis Daten von einem Datenträger in den Hauptspeicher gelesen sind?

c) Ein Datenträger verwendet Blöcke mit einer Größe von 1 MB und bietet insgesamt 5 TB Speicherplatz. Ein FAT-Eintrag koste beim verwendeten Dateisystem 10 Byte.

Beantworten Sie unter diesen Rahmenbedingungen die folgenden Fragen:

- Wie viel Platz belegt eine Textdatei mit 50 kB Inhalt auf dem Datenträger?
- Wie viel Platz benötigt die gesamte FAT?
- Liegt die FAT auf dem Datenträger oder im Hauptspeicher?
- Wie verändert sich der Platzbedarf, wenn man stattdessen I-Nodes nutzt?

d) Welchen Vorteil hat NTFS gegenüber einer FAT-Verwaltung?

e) Gegeben sei die folgende Datenstruktur für einen FAT-Eintrag als Pseudocode:

```
STRUCT fatentry

    INT file_block_address       // physische Addr. des Blocks
                                 // auf dem Datenträger, ist ein
                                 // Vielfaches der block_size.
    CHAR file_name[16]
    INT timestamp_changed
    INT timestamp_created
    INT flags
```

Die gesamte FAT liege als Array `fat` wie folgt vor:

```
fatentry fat[65536]
```

Ein einzelner Block auf dem Datenträger sei 1 kB groß:

```
INT block_size = 1024
```

Beantworten Sie unter diesen Vorgaben die folgenden Fragen:

- Wie viel Speicherplatz ist auf dem Datenträger vorhanden?
- Angenommen, Sie kennen den ersten `fatentry` einer Datei. Schreiben Sie ein Programm (nicht-obskure Programmiersprache oder Pseudo-Code), das alle zur Datei zugehörigen Block-Startadressen ausgibt.

f) Gegeben sei die folgende Datenstruktur für einen I-Node-Eintrag als Pseudocode:

```
STRUCT inode

    INT file_block_addresses[8]    // 0 if no block is linked.

    CHAR file_name_[16]
    INT timestamp_changed
    INT timestamp_created
    INT flags
    REFERENCE_TO(inode) one_indirection[16]
    REFERENCE_TO(inode) two_indirections[16]
    REFERENCE_TO(inode) three_indirections[16]
```

Beantworten Sie unter diesen Vorgaben die folgenden Fragen:

- Angenommen, Sie kennen den ersten `inode` einer Datei. Schreiben Sie ein Programm (nicht-obskure Programmiersprache oder Pseudo-Code), das alle zur Datei zugehörigen Block-Startadressen ausgibt.
- Welchen Vorteil bietet die Aufgliederung in 1-, 2- und 3-stufige Indirektionsknoten, statt einen gewöhnlichen Baum mit beliebiger Tiefe zu erlauben?